

Capstone

JANE STREET MARKET PREDICTION

<https://www.kaggle.com/c/jane-street-market-prediction>

Prathyusha Sathineni

Introduction

Jane street market prediction is a Kaggle competition. In this trading challenge, we must predict the action value for a trading opportunity. For more information about the Jane street market prediction, please go through the above-provided link. In this project, we will be implementing different machine learning models on the real-time market dataset. This model is for the prediction of action value for a trading opportunity. Finally, we will be evaluating the models based on the confusion matrix and accuracy metrics. Therefore, the main goal of this project is to find out the most accurate machine learning model for predicting the action value.

About Dataset

It is a 6GB dataset that contains an anonymized set of features that represents actual stock market data. Each row in the dataset represents a trading opportunity, for which we will be predicting an action value: 1 to make the trade and 0 to pass on it.

Following is the list of files we will be using for our analysis

- **train.csv** - the training set, contains historical data and returns. It contains 2390491 rows × 138 columns. The train.csv has the following attributes:
 - feature_{0...129} - anonymized features.
 - resp, resp_{1,2,3,4} - represents returns over different time horizons.
 - weight - represents weight associated with each trade
 - date - represents the day of trade
 - ts_id - represents time ordering
- **example_test.csv** - a mock test set that represents the structure of the unseen test set. We will not be using this test dataset directly to test the accuracy of the

model because this is just a sample test data provided by Jane Street Market, and it has all the attributes same as that of train.csv, except resp values. It contains 15219 rows × 133 columns only.

- **features.csv** - metadata about the anonymized features. It contains 130 rows × 29 columns. It provides information about anonymized tags {0,.....,28} - Boolean values, for each feature.

Data Preprocessing

Data preprocessing is an important task. It is a data mining technique that transforms raw data into a more understandable, useful, and efficient format [1]. I have followed the below-mentioned steps for data preprocessing (train.csv file).

- **Step 1: Handling Null Values** – Figure 1 shows the number of null values present in each column. From the figure, we can observe that the maximum number of null values a column could have is 395535, and the minimum is 1. From analysis, it is observed that if we remove records with null values, the models are leading to overfitting. Therefore, the best way to handle the null values is by replacing them with the mean.

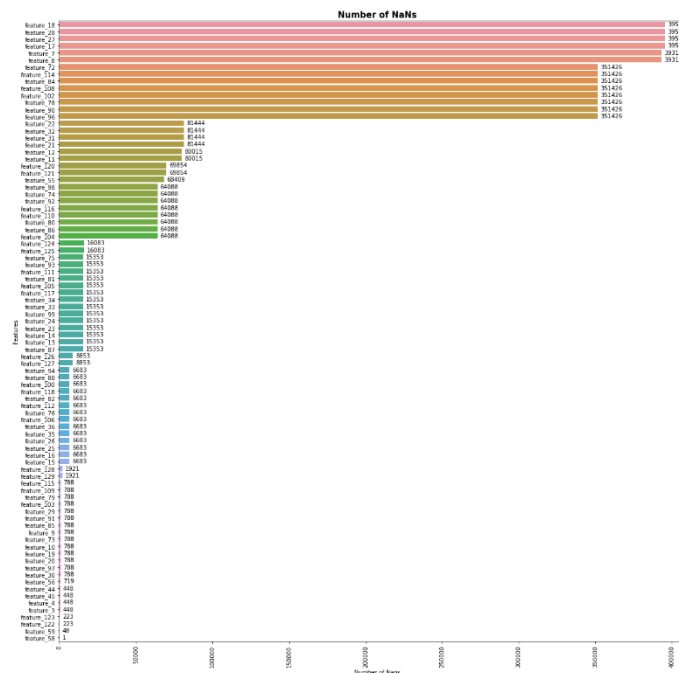


FIGURE 1 : NUMBER OF NaNs

- **Step 2: Handling weight = 0 records** - Figure 2 shows the percentage of records with non-zero weight and zero weight. Based on the description provided in the Kaggle competition, weight zero records indicate gain from that trade is 0. Therefore, we can drop records with zero weight.

Percentage of Zero Weight and Non-Zero Weight records

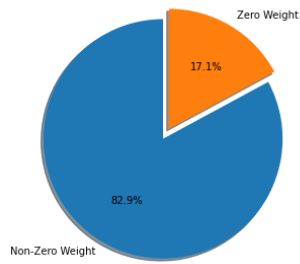


FIGURE 2: PERCENTAGE OF WEIGHTS

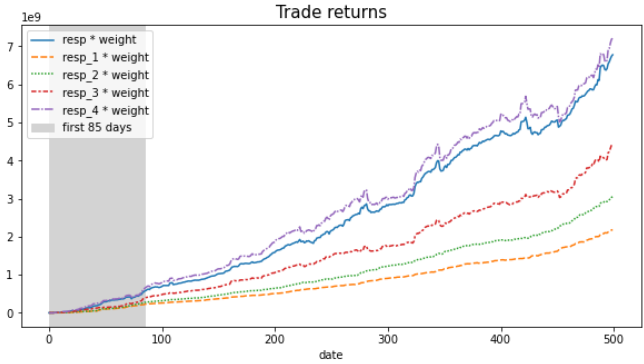


FIGURE 3: TRADE RETURNS

- **Steps 3: Data reduction based on trade returns** - In the dataset, we have transactions for 500 days. The description of data states that each trade has an associated weight and resp, which together represents a return on the trade [2]. From figure 3, we can observe that trade returns started increasing drastically after 85 days. Therefore, we can exclude records before 85th day from the dataset.

Feature Selection

We will start the feature selection process first by exploring the features.csv file. As mentioned earlier, the file contains information about anonymized tags {0,...,28} - Boolean values for each feature. For analysis purposes, I converted the true/false value into the 1/0 format. After converting, I have calculated the count of tags for each feature and the number of features each tag has as well. The visualization of the tag count for each feature is in Figure 4, and the visualization for the features count for each tag is shown in Figure 5.

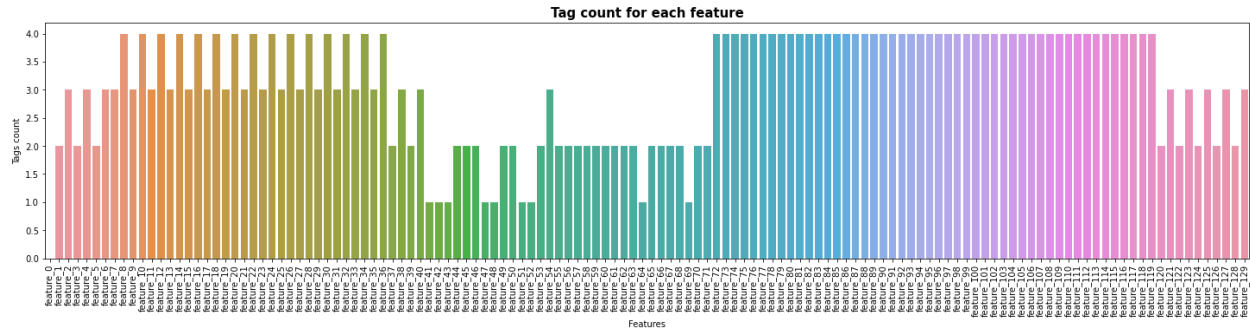


FIGURE 4: TAG COUNT

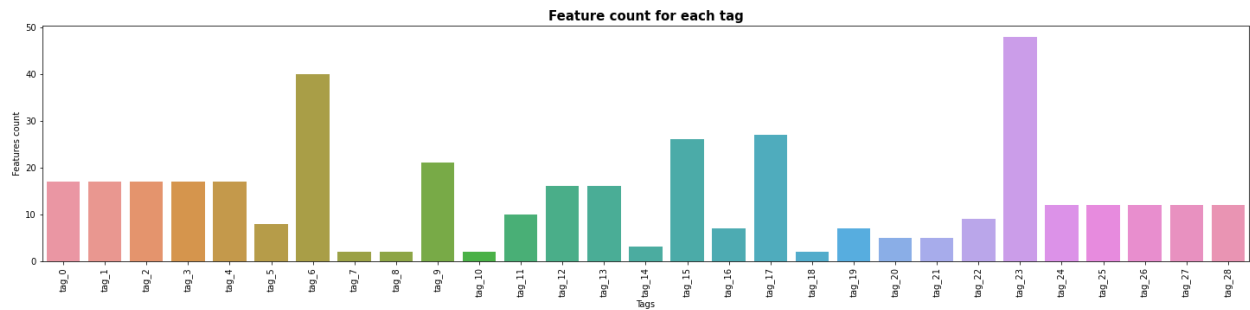
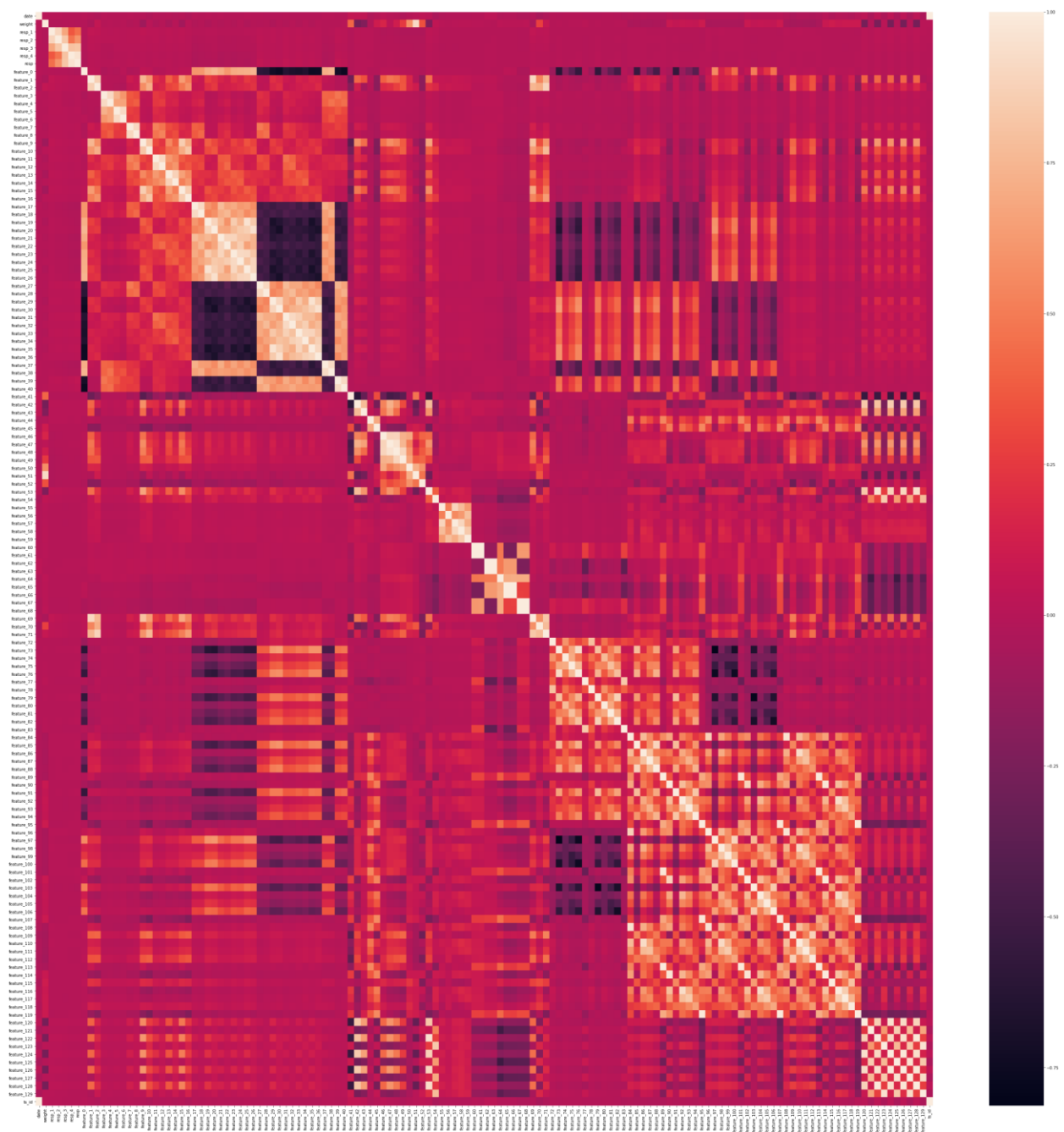


FIGURE 5: FEATURES COUNT

Interesting observations from figures 4 and 5 :

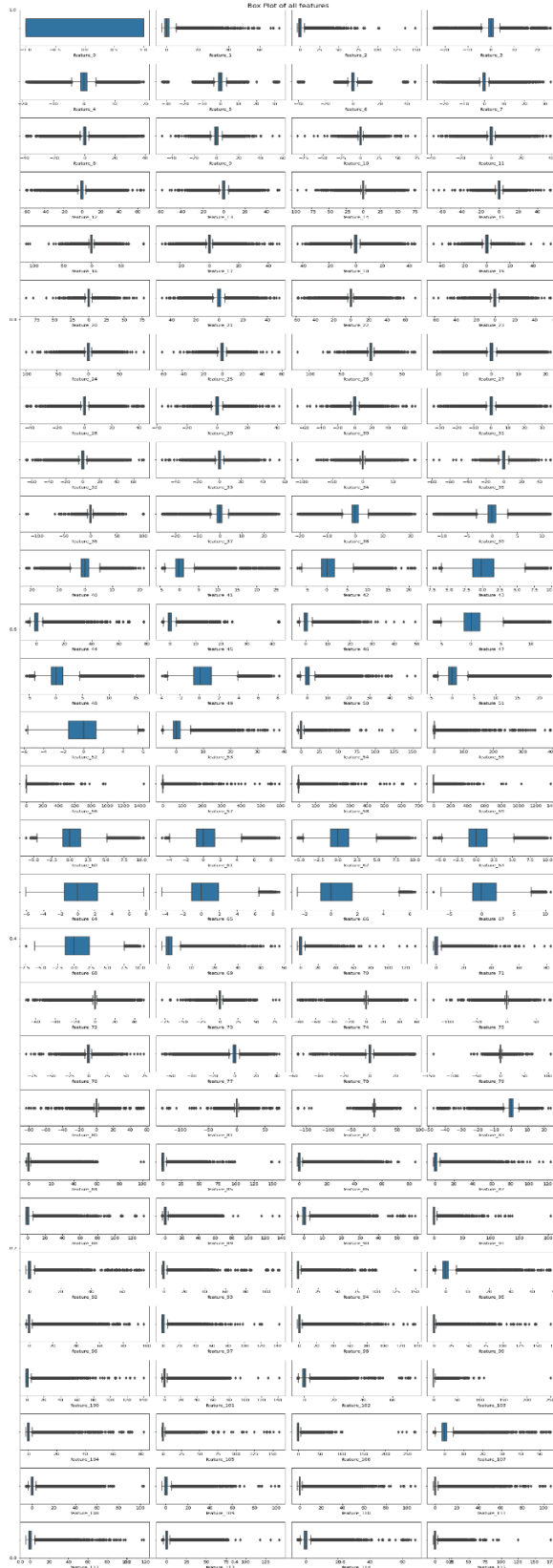
- The feature_0 has 0 tags.
- feature_{8,10,12,14,16,18,20,22,24,26,28,30,32,34,36,72,.....,119} has 4 tags
- feature_{2,4,6,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,38,40,54,121,123,125,127,129} has 3 tags
- feature_{1,3,5,37,39,44,45,46,49,50,53,55,...,63,65,66,67,68,70,71,120,122,124,126,128} has 2 tags
- remaining features, feature_{41,42,43,47,48,51,52,64,69,} has only 1 tag
- Approximately 40+ features have tag_6 and tag_23.
- 20+ features have tag_9, tag_15 and tag_17
- The First 5 tags and last 5 tags have the same number of features. 17, and 12 respectively.

Based on the above observations, it is hard to select important features. Therefore, I have created a heatmap to check the correlation between the features.



From figure 6, we can observe that there are a lot of multicollinearities, and it's hard to get the important features from these clusters. Therefore, I have created a grid with a box plot for all the features to check the outliers. Figure 7 shows the grid with a box plot for all the features.

FIGURE 7: OUTLIERS



However, we can observe that all the variables have many outliers with larger values, and without domain knowledge, it is inappropriate to eliminate attributes based on outliers. Hence, it will be apt to select the features using the random forest feature importance method.

To go forward with the random forest feature selection process, we need a target variable. In this project target variable is nothing but the action value. As mentioned earlier, if the action value is 1, it means to make a trade, or else if it is zero, pass it on. So, as a part of the hit and trial analysis process, I have used two approaches for creating the target variable. The approaches are as follows:

Approach 1 : If $\text{resp} > 0$ then $\text{action} = 1$, else $\text{action} = 0$

Approach 2: In the description on Kaggle, they stated that each weight is associated with resp value. Therefore, I have created an action variable using the following condition: if $(\text{resp} * \text{weight}) \& (\text{resp}_1 * \text{weight}) \& (\text{resp}_2 * \text{weight}) \& (\text{resp}_3 * \text{weight}) \& (\text{resp}_4 * \text{weight}) > 0$ then $\text{action} = 1$ else $\text{action} = 0$.

Using the above target variable approaches, when we extract the top 20 features using random forest trees, we get features as shown in figure 8 and figure 9.

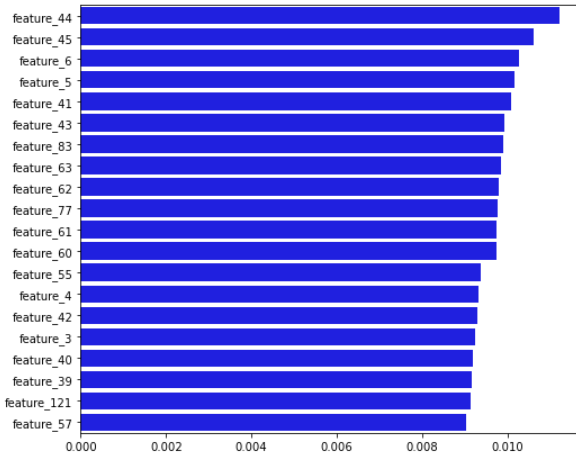


FIGURE 8: IMPORTANT FEATURES USING APPROACH 1

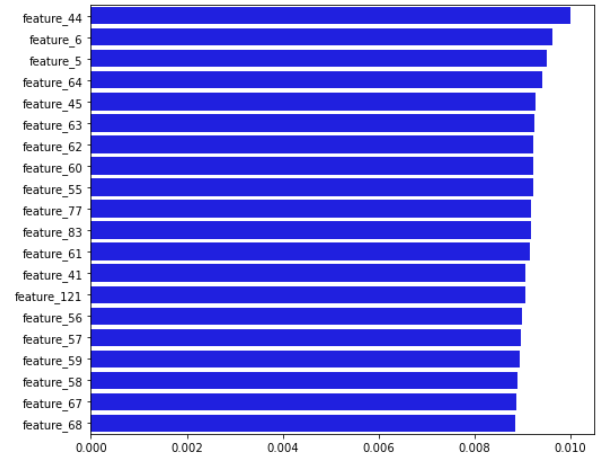


FIGURE 9: IMPORTANT FEATURES USING APPROACH 2

Interesting observations between important features of 2 approaches:

1. feature_44 is the first important variable.
2. feature_6 ranks in the top 3 in both.
3. There are 14 same features.

Data Partitioning

In the step data partitioning, we partition the preprocessed train.csv file into training and test datasets. The preprocessed data set contains only important features and the target variable. I have considered randomly shuffled 80% of the data as a training dataset and the remaining 20% of the data as a test set.

Model Evaluation Metrics

The choice of metric depends on the type of model and the implementation plan of the model [4]. As mentioned earlier in the report, our target variable is the action value. The action value is a binary value 0 or 1. Therefore, we will be using the classification models for predicting the action value. Since we will be using classification models for prediction, we will be using the confusion matrix as metrics for model evaluation. The confusion matrix is another metric that is often used to measure the performance of a classification algorithm [3]. The matrix compares the actual target values with those predicted by the machine learning model [5]. The confusion matrix report provides us with the dimensions like accuracy, precision, recall, and f1-score. In this project, we will be mainly using the accuracy dimension for the model evaluation. Accuracy gives the proportion of the total number of predictions that were correct [4].

Models

In this section, we will be discussing the models we have used for predicting the action value. As mentioned in the model evaluation metric section, I have used binary classification models. The classification models used in this project are Multiple layer perceptron, Decision tree, and Random Forest trees. 2 Approaches are used to run the models. Table 1 provides the information about the confusion matrix and accuracy obtained for the respective models when the model is tested with the test dataset.

	Approach 1	Approach 2
Action value	If $\text{resp} > 0$ then action = 1, else action = 0	If $(\text{resp} * \text{weight}) \& (\text{resp}_1 * \text{weight}) \& (\text{resp}_2 * \text{weight}) \& (\text{resp}_3 * \text{weight}) \& (\text{resp}_4 * \text{weight}) > 0$ then action = 1 else action = 0.
Multiple Layer Perceptron	Confusion matrix: $\begin{bmatrix} 10019 & 615424 \\ 10206 & 621483 \end{bmatrix}$ Accuracy: 50.23%	Confusion matrix: $\begin{bmatrix} 924363 & 0 \\ 332769 & 0 \end{bmatrix}$ Accuracy: 73.53%
Decision Tree	Confusion matrix: $\begin{bmatrix} 374822 & 250621 \\ 252364 & 379325 \end{bmatrix}$ Accuracy: 59.98%	Confusion matrix: $\begin{bmatrix} 677816 & 246547 \\ 225887 & 106882 \end{bmatrix}$ Accuracy: 62.42%
Random Forest Tree	Confusion matrix: $\begin{bmatrix} 334236 & 291207 \\ 310638 & 321051 \end{bmatrix}$ Accuracy: 52.13%	Confusion matrix: $\begin{bmatrix} 924363 & 0 \\ 332769 & 0 \end{bmatrix}$ Accuracy: 73.53%

TABLE 1: MODELS AND ITS ACCURACY

Model Selection

Model selection is the process of choosing one among many candidate models for a predictive modeling problem. In this section, we will be selecting the best model based on the accuracy metrics. From table 1, we can notice some interesting observations. The

accuracy of all the models is high in approach 2 when compared to approach 1. We can observe that in approach 1, the decision tree is the most accurate model. Whereas in approach 2, it is the model with the lowest accuracy. One more interesting observation is that in approach 2, both the model's multiple layer perceptron and random forest tree have the same accuracy. Multiple layer perceptron and random forest tree of approach 2 stand out to be the highest accuracy models among all six models with 73.53% accuracy. Since multiple layer perceptron, nothing but neural network, and random forest trees are the two widely used models. It is hard to pick one when accuracy is the same. Therefore, we will be choosing the model based on the dataset size. Random Forests often have little performance gain when a certain amount of data is reached, while Neural Networks usually benefit from large amounts of data and continuously improve the accuracy [7]. Hence, from the above discussion, we can consider the 2nd approach as the best approach and multiple layer perceptron to be the most accurate model that can be used for predicting the action value.

Conclusion

From the above discussion, we can conclude that approach 2 gives better accuracy for all three models. Among these three models, multiple layer perceptron would be the best model to predict the action value. The hardest part of this project was handling the 5 GB of data which needed a lot of preprocessing. To run various models on 5 GB data, we need more memory, and the system crashed several times. To overcome this problem, in the future, we can use AWS and try various models. One more hardest part was the feature selection process. Since this is competition data, all the features were anonymized, so it's tough to figure out which variable is useful for analysis and which is not required. In the future, maybe the feature selection process can be improved by using any of the 3 approaches listed below:

- By doing more analysis on the clusters formed in the correlation heatmap.
- By seeing the feature count plot, we can understand that each tag has a set of features. Possibly we can do cluster analysis by creating a cluster for each tag. Therefore we have to analyze 28 groups.

- By doing more research and identifying the relationship between feature and tags. So that by using the tags count of each feature possibly we can extract important features.

References

- [1] <https://towardsdatascience.com/data-preprocessing-e2b0bed4c7fb>
- [2] <https://www.kaggle.com/c/jane-street-market-prediction/data>
- [3] <https://medium.com/hugo-ferreiras-blog/confusion-matrix-and-other-metrics-in-machine-learning-894688cb1c0a>
- [4] <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>
- [5] <https://www.analyticsvidhya.com/blog/2020/04/confusion-matrix-machine-learning/>
- [6] <https://machinelearningmastery.com/a-gentle-introduction-to-model-selection-for-machine-learning/>
- [7] <https://blog.frankfurt-school.de/wp-content/uploads/2018/10/Neural-Networks-vs-Random-Forests.pdf>
- [8] <https://www.kaggle.com/wendelfariaslopes/easy-and-simple-xgboost-good-score>
- [9] <https://www.kaggle.com/ashagutlapalli/time-series-analysis-jane-street-market-eda>
- [10] <https://www.kaggle.com/liuzhangyu/cs-random-forest>
- [11] <https://www.kaggle.com/liuzhangyu/cs-mlp>
- [12] <https://www.kaggle.com/huzzefakhan/nn-model-for-jane-street>
- [13] <https://www.kaggle.com/andreasthomasen/preprocessing-and-feature-selection>
- [14] <https://www.kaggle.com/bosbos/stock-id-feature>
- [15] <https://www.kaggle.com/a763337092/neural-network-starter-pytorch-version>
- [16] <https://leejaeka.github.io/jaekangai/python/eda/jane%20street/kaggle/visualization/big%20data/2021/01/23/JaneStreet-Copy1.html>